ゼロヒューマン会社の運営本

エージェントが仕事を引き継ぎ、結果を確かめるための実務ガイド

Anicca

- はじめに
- 1. 無人という目標と、現在の運用
- 2. 成果を一つに決める
- 3. 役割と所有権を分ける
- 4. 短い実行を積み重ねる
- 5. 事実を残して再開する
- 6. 重複した外部操作を防ぐ
- 7. 認証と予算の境界を置く
- 8. 失敗から次の一手を決める
- 9. 販売と納品をつなぐ
- 10. 日次の数字で順序を変える
- 11. 最初の一週間を組み立てる
- 12. 引き継げる会社にする
- 付録A. そのまま埋める一枚のタスク
- 付録B. 毎日の短い報告
- 付録C. テンプレート版の内容
- 出典、制作範囲、利用条件

はじめに

夜にエージェントへ仕事を渡した。朝になると、長い説明と「あと一歩です」という報告が届く。ページはできているが買えない。送信ボタンは押したが送信済み一覧にはない。次に起きたエージェントは、同じ調査を最初から始める。

この本が扱うのは、その「あと一歩」を誰が、何を根拠に、どう閉じるかである。

Anniccaでは、ソフトウェア、デジタル商品、配信、受託業務などを複数のエージェントへ分けて運用する。そこで繰り返し問題になるのは、文章を生成する能力より、現在の所有者、実際の結果、再実行の可否、次の一手を保つことだ。本書は、運用ルールと商品・ループの台帳を読み、他の環境でも使える形へ書き直した実務ガイドである。顧客情報、アカウント情報、秘密値、内部の連絡先は含めない。

「ゼロヒューマン」は到達を目指す運営方針を指す。人間の操作が不要になった会社、無停止のサービス、利益の出る事業が完成したという実績表示ではない。認証の失効、サービス側の審査、接続障害、権限不足は今も起こる。エージェントが操作しているというだけで、契約、責任、本人確認の条件がなくなることもない。

本書を読んだあとに作るものは、巨大なエージェント組織ではない。「一つの仕事が、別の担当に変わっても、二重実行せずに結果まで進む」小さな運用である。

読み方と使い方

最初に一つだけ、繰り返している業務を選ぶ。既存の入力があり、結果を外から確認できる仕事が向く。たとえば、購入後のデジタル納品、公開済みページの保守、決まった条件の問い合わせ分類である。最初から営業、開発、経理、公開を一つにまとめない。

第1章から第4章で仕事の単位と担当を決める。第5章から第8章で、再開、重複防止、権限、失敗対応を設計する。第9章と第10章で販売と数字をつなぎ、第11章の一週間の手順で小さく稼働させる。第12章は担当が変わるときに使う。

本文中の注文ID、金額、顧客、実行記録は、商品価格の説明を除き、仕組みを示すための架空例である。Anicca の売上や稼働実績として扱わない。設定例は完成した実行システムではなく、利用するツールの仕様に合わせて埋める雛形である。

1. 無人という目標と、現在の運用

無人化を考えるとき、「何人のエージェントがいるか」から始めると、仕事より先に組織図が増える。まず現在、人が閉じている作業を一つずつ書く。

たとえばデジタル商品の販売には、本文を書く、価格を登録する、購入ボタンを置く、支払いを確かめる、適切なファイルを渡す、受け取れなかった購入者を助ける、という別々の作業がある。本文の生成だけを自動化しても、販売全体が自動になったわけではない。

それぞれに「入力」「操作」「確認先」を置く。確認先は、操作した側の文章以外に置く。支払いなら決済事業者の記録、公開なら公開URL、送信なら送信済みの記録、納品なら受領または取得できたファイルである。

作業	入力	外から確認するもの
商品を作る	購入者の用途と素材	販売するファイルの内容
価格を公開する	商品IDと価格	公式カタログと購入画面
納品する	支払い済み注文	注文に対応した取得結果
問題を直す	失敗した注文と記録	元の失敗条件での再確認

ここで、理想と現在を同じ列に書かない。「自動公開する予定」と「公開済み」は別の状態である。予定、実装、稼働、外部の結果を分けておけば、次の担当は足りない一段から再開できる。

Anicca の運用資料にも、担当が登録されているのに稼働確認が足りない仕事や、コードの修正と外部サービスへの反映が分かれている仕事が残る。本書はその差を隠して成功物語にするためのものではない。差を見つけたとき、仕事を捨てずに次へ進めるための手順を扱う。

ここで決めること： 選んだ業務の最後に、何が見えれば利用者の目的を満たしたと言えるかを一文にする。「決済事業者で支払い済みの注文について、その版のPDFを取得できる」ほど具体的に書く。

2. 成果を一つに決める

「売上を増やす」「サイトを改善する」「AIで自動化する」は方向であり、一回の実行に渡す仕事ではない。実行担当へは、今ある事実と、今回変える範囲と、終わりの条件を渡す。

悪い依頼は「販売ページをいい感じにしておいて」。良い依頼は「既存の商品Aについて、価格29ドルの購入導線を公開し、公式価格と購入画面の一致を確認する。商品Bや配信スケジュールは変更しない」である。どちらも裁量は残るが、後者は結果を判定できる。

一つのタスクを、次の六つで書く。

1. 目的: 利用者が得る結果。
2. 現在の根拠: URL、ファイル、注文、失敗した実行。
3. 範囲: 触る商品、環境、ファイル、アカウント。
4. 外部操作: 公開、送信、価格変更などの具体的な対象。
5. 完了条件: 何を取得して確かめるか。
6. 次に残すもの: 結果、未確認、次の一手。

完了条件は、作業の途中で安心するために無制限に増やさない。文言の誤字を直す依頼に、購入、全画面テスト、デザイン刷新まで足す必要はない。一方、販売可能にする依頼に、決済設定と納品を含めずに「ページ完成」とするのも足りない。必要な確認は、依頼の成果が失敗する条件から選ぶ。

順序も成果から決める。購入ボタンが壊れているのに、先に宣伝画像を十枚作っても販売は進まない。支払いができても納品ファイルがないなら、先に売るほど問い合わせが増える。入力、商品、価格、決済、納品のつながりを一度紙に書くと、依存関係が見える。

例として、既存商品に上位版を足すなら、通常版の購入者へ上位版のファイルを返さない条件を先に決める。価格だけを作ってからアクセス制御を考えるより、検証の対象が明確になる。

ここで作るもの: 一枚のタスク正本。会話の最後の発言だけに最新条件を残さず、担当が最初に開く場所へ保存する。仕様の正本がすでにあるなら、同じ内容の第二の正本は増やさない。

3. 役割と所有権を分ける

役割は「何が得意か」。所有権は「今、この対象を変えてよいのは誰か」である。この二つを混ぜると、全員が助けようとして同じ価格を変更したり、同じ問い合わせへ二度返信したりする。

小さく始めるなら、主担当が目的、順序、完了判定を持つ。実装担当は限定されたファイルや設定を変える。別の担当へ渡す実益がなければ、一人のエージェントが両方を担ってよい。人数を増やすこと自体を目標にしない。

外部対象には一人の現在所有者を置く。アカウント、商品、注文、リリース、ブラウザの認証状態ごとに考える。アカウントへログインできることと、その瞬間に変更してよいことは同じではない。

対象	現在所有者	許される操作	返す証拠
商品Aの価格	商品担当	指定した価格の変更	商品ID、公式価格
商品Aの投稿	配信担当	指定媒体への投稿	公開投稿URL
注文Xの納品	納品担当	注文に対応した送付	注文ID、納品記録

委任では、担当者の名前だけを残さない。目的、所有範囲、禁止事項、根拠、検証、報告形式を渡す。「お願いした」状態と「相手が着手した」状態を分ける。メッセージを送れたことは、相手の実行開始を証明しない。

たとえば商品担当が価格を変えている間に、実装担当が古い設定から自動同期すると、変更は消える。担当名が分かっていても、同じ設定を二つの経路から書いていれば競合する。所有権の表には、人やエージェントだけでなく、自動処理の名前と書き込み先も含める。

担当交代は、前の担当が何を外部へ出したかを読むところから始める。未確認の送信があれば、公式記録を取得するまで同じ送信を繰り返さない。交代は仕事の初期化ではない。

ここで決めること： 一番壊れやすい共有対象を一つ選び、現在所有者と書き込み経路を一つずつ記録する。

4. 短い実行を積み重ねる

長時間のエージェント実行は、途中で利用枠、接続、認証、文脈を失う可能性がある。そこで一回の実行を、途中から再開しやすい単位へ分ける。

基本の形は「状態を見る、今回の一件を選ぶ、実行する、結果を確かめる、保存する」である。毎回大きな計画を立て直す必要はない。前回の記録を読み、先頭の未完項目を進める。

一件の大きさは時間だけで決めない。結果を独立して確かめられ、次の実行へ渡せるかで決める。販売ページなら、「見出しを考えた」は弱い区切りである。「商品名と価格と納品形式を正本へ確定した」なら次の担当が使える。「価格を登録し、商品IDと公式価格を取得した」ならさらに外部へ進んでいる。

定期実行と仕事の判断も分ける。スケジューラは起動時刻、実行環境、終了条件を持つ。エージェントはその回に進める仕事を選ぶ。エージェントに永久に起き続ける責任まで持たせると、停止したときに誰も起こせなくなる。

起動時に確認するのは、前回の終了、現在所有者、未確認の外部操作、入力の更新、予算の残りである。前の実行がまだ同じ対象を操作中なら、同じ対象を奪わず、独立した作業へ進む。ロックを待つだけの実行を増やす必要もない。

外部の公開窓が閉じているとき、その媒体の公開だけを待ち状態にする。原稿の確認、価格の照合、既存の問い合わせ対応まで止めない。待ち状態には、何を待つのか、再開条件、次に確認する時刻、誰が再開するかを残す。

実行の終了メモの例:「商品Aの価格登録を完了。公式価格は29ドル。公開ページへの反映は未完。次回は既存ボタンのリンク先だけを変更し、公開URLから購入画面へ到達するか確認する。」これなら、次回は価格登録をやり直さない。

5. 事実を残して再開する

会話の要約だけでは、再開に足りないことがある。「直した」「公開した」「売れた」は、何をどこまで確かめたかが欠ける。記録は、次の担当が同じ判断を再現できる程度に具体的にする。

まず、事実と推論を分ける。HTTP　200は応答を得た事実である。購入者が正しいファイルを受け取れたかは、別の確認が要る。ダッシュボードの売上表示と、銀行への入金も違う段階である。

次の段階を一つの言葉でまとめない。

段階	記録できる表現
操作した	価格変更を送信した
保存を確認した	公式カタログで価格を取得した
公開を確認した	公開URLの表示価格が一致した
支払いを確認した	注文が paid になった
納品を確認した	その注文で該当版を取得できた

記録には、実行ID、所有者、対象の発生ID、使用したリリース、工程、コマンドまたは操作、終了コード、外部効果、確認結果、公式記録ID、根拠の保存先、エラー種別、再試行の可否、次の一手を残す。小さな仕事で全項目が埋まる必要はない。該当しないものと、まだ分からないものを分ける。

```
{
  "run_id": "run-example-001",
  "owner_id": "delivery-worker",
  "occurrence_id": "order-example-123",
  "phase": "delivery_readback",
  "effect": "download_allowed",
  "readback": "matching_edition_received",
  "provider_receipt_id": "receipt-example-123",
  "error_class": null,
  "retryable": false,
  "next_action": "close_order"
}
```

記録そのものへ秘密値を入れない。認証情報の場所は示しても、値を本文、チャット、ログ、Gitへ複製しない。取得したデータに個人情報があるなら、必要な判定結果と識別子だけを残す。

ここで作るもの: 直近の一件について、「操作」「公式確認」「利用者の結果」をそれぞれ一行で書く。取得していない段階は未確認のままにする。

6. 重複した外部操作を防ぐ

同じ調査を二度することと、同じ注文に二度請求することは、同じ失敗ではない。外部へ出る操作には、その一回を識別する鍵と、再試行の判断を置く。

たとえばネットワークの切断が起きたとする。送信側は応答を受け取っていない。しかし、相手のサービスでは注文作成が済んでいるかもしれない。この状態で同じ作成をもう一度送ると、重複する可能性がある。

処理の前に、対象、操作の種類、一回の発生を結ぶ識別子を決める。定期実行なら「対象アカウント、対象日、処理の種類」。注文なら「注文ID、納品の種類」。本文の少し違う言い換えや、担当の交代で別の識別子にならないようにする。

サービスが冪等性キーを提供するなら、その契約に従って使う。提供しない場合でも、手元の台帳に claim、送信、確認を残せる。ただし手元の台帳だけで、相手側に結果が存在しないと断定しない。

台帳の状態	次にすること
まだ実行していない	所有権を確保して実行
実行中	同じ対象を別担当で実行しない

台帳の状態	次にすること
応答が不明	公式記録を照合する
完了を確認済み	再送せず結果を返す
失敗が確定	原因を直し再試行条件を確認

不明な外部操作を保留する境界を、本書では effect fence と呼ぶ。これは仕事を終える理由ではない。次の仕事は、再送ではなく、公式記録を探して照合することである。

作成した本人の文章は、その照合の代わりにならない。「送信に成功しました」とログにあっても、相手の送信済み一覧、注文一覧、公開URLを確認する。確認できなければ、何が欠けているかを具体的に残す。

練習:「APIへ送った直後に通信が切れた」ケースで、再送より先に取得する公式記録を一つ書く。見つかった場合と、失敗が確定した場合の次の動きを分ける。

7. 認証と予算の境界を置く

エージェントの裁量を広げても、認証情報の所有者やサービスの規約が変わるわけではない。共有のブラウザで別の人のアカウントが開いていた、という状態を作らないことが先である。

認証情報は、管理する秘密の保存先を一つ決める。サービス名で対象を完全に照合し、別サービスの似た名前のキーを使わない。新しく作った秘密値は、その保存先に保存する。チャット、タスク説明、テンプレート、公開リポジトリには入れない。

ブラウザを使う場合は、サービス、アカウント、プロファイル、現在所有者を結び付ける。現在のログイン状態は、開いている画面やサービスの応答で確認する。古い設定ファイルがあるだけで、今も正しいアカウントだと判断しない。

予算は「気を付ける」という文章だけでは管理しにくい。通貨、対象期間、上限、現在の使用量、金額の根拠を分ける。実測値がないときは unknown とし、ゼロにしない。

```
{
  "currency": "USD",
  "window": "daily",
  "limit": null,
  "spent_verified": null,
  "source": null,
```

```
  "on_limit": "stop_new_spend_and_reconcile"
}
```

この例の `null` は、無料で使ってよいという意味ではない。所有者が有効な上限と根拠を設定してから使う項目である。支出の権限と、口座から資金を動かす権限も同一視しない。

外部の本人確認、CAPTCHA、決済事業者の確認が現れたら、サービスが提供する通常の手続きへ進む。境界を回避するために、別人のアカウントや秘密値を流用しない。必要な条件が欠ける場合は、どの条件かを記録し、独立して進められる仕事を続ける。

ここで決めること: 選んだ一業務について、認証の対象、秘密の正本、ブラウザ所有者、支出上限を確認する。値そのものをタスクへコピーせず、照合結果だけを残す。

8. 失敗から次の一手を決める

「失敗しました」「ブロックされています」で終えると、次の実行は同じボタンをもう一度押すしかなくなる。失敗を、どの境界で何が分からなくなったかに分ける。

入力が違うのか。呼び出し元が古いのか。設定が読み込まれていないのか。通信が届かないのか。相手が拒否したのか。送信は済んだが結果を読めないのか。この区別によって、必要な操作は変わる。

同じ失敗をもう一度実行する前に、観測を一つ増やす。たとえば、読み込んだリリース、実際の引数、秘密値を除いた設定名、HTTPステータス、相手のエラー種別、対象IDを記録する。コードの不具合なら、元の失敗を表す小さいテストを先に失敗させる。

例として、購入画面が開かないとする。ボタンのリンクが空なのか、価格が無効なのか、決済アカウントが支払いを受け付けていないのかで修正先は違う。見た目を作り直す前に、クリック先と公式カタログの有効状態を照合する。

403も万能な診断名ではない。認証不足、所有権不足、サービス側の制約などが考えられる。応答を取得した経路と、試した別経路を残す。同じアカウントの公式APIが読めるなら、画面を取れないという理由で既存情報の確認まで諦めない。

自分が所有するコード、設定、読み取り不足が原因なら、その場で直す。別の所有者の対象なら、具体的な対象と必要な結果を渡す。引き継いだだけで完了にせず、担当の着手と返す証拠を確認する。

外部の審査待ちなど、今は変更できない条件もある。そのときは「審査中の商品Aを重複作成しない。公開版は維持する。審査結果を同じ商品IDで確認する」のように、保つ対象と再開条件を書く。

失敗報告の型:「確認した事実、原因として確定したこと、未確定、今回の変更、検証結果、次の一件」。原因がまだ推論なら、その言葉を残す。

9. 販売と納品をつなぐ

販売可能な状態は、見出しと価格が並んでいる状態より一段深い。買うもの、支払い、受け取り方が同じ契約を指している必要がある。

商品には、誰が何に使い、何を受け取るかを書く。上位版があるなら、通常版との差をファイル単位で説明する。「特別な価値」より、「編集可能な役割契約と台帳のZIPが付く」の方が判定できる。

この本の二つの版も、同じ本文を使う。本文版はPDFとMarkdown、テンプレート版はそれに編集用ZIPを加える。実行代行、個別導入、将来の売上、無期限の個別サポートは含めない。販売する以上、公開ページと実際の納品物を合わせる。

決済では、商品ID、価格ID、金額、通貨、一回払いか継続かを公式記録で照合する。購入ボタンの文字だけを変えても、古い価格へのリンクが残っていれば、実際の請求は変わらない。

納品側は、支払い済みか、対象の商品か、その版か、使用する環境かを確かめてからファイルを返す。URLに session_id があるだけで購入済みと扱わない。決済事業者が返す状態を読む。

条件	期待する動作
支払い前の注文	ファイルを返さない

条件	期待する動作
別商品の注文	ファイルを返さない
通常版の注文で上位版を要求	上位版を返さない
テスト決済を本番で使用	ファイルを返さない
正しい支払い済み注文	対応する版を返す

実決済をしていない検証は、その限界を示す。本番Checkoutへ到達できたこと、ローカルの支払い済みfixtureで正しいファイルを返せたこと、実際の購入者の支払いと納品は、三つの異なる証拠である。自分で試した請求を事業の顧客売上へ混ぜない。

本文や雛形を公開リポジトリへ置く場合は、有料版で何を売っているかも明確にする。ソースの独占利用権ではなく、整形したPDFとまとめたパッケージを売るなら、その通りに書く。隠れた条件で価値を作らない。

10. 日次の数字で順序を変える

自律運用の数字は、起動回数と生成量だけでは足りない。百回実行しても、同じ障害を繰り返しているなら利用者の結果は進まない。

一つの仕事について、実行数、確認できた前進、外部効果、費用、残る不明を分けて記録する。販売なら、ページの閲覧、購入画面への移動、支払い、納品、返金を分ける。それぞれ違う場所が壊れうる。

売上は、公式な支払い記録を根拠にする。閲覧、いいね、ダウンロード希望、テスト決済を足さない。一回払いと継続売上を混ぜず、MRRへ単発売上を入れない。通貨が異なるなら、換算に使った値と時点も区別する。

記録できなかったものは unknown とする。測定できなかった売上をゼロにすると、データ不足と本当のゼロが同じになり、優先順位を誤る。

順序変更は、その日の気分ではなく、観測した詰まりから行う。閲覧はあるのにCheckoutが開かないなら、配信を増やすより購入導線を直す。支払いはできるのに納品が止まるなら、販売促進より先に納品を直す。購入導線が通っているが閲覧がないなら、媒体と商品に合う実例を使って流入を作る。

変更するときは、旧順序、新順序、理由、現在進めている一件を同じ差分に残す。実行中の注文や公開操作は中断・重複させない。

日次報告の例:「確認済み支払い2件、納品2件。広告費は公式明細未取得でunknown。明日は新しい商品を足さず、本文版とテンプレート版の違いを購入ページ上で分かりやすくする。」この数字は説明用の架空例であり、実売上ではない。

11. 最初の一週間を組み立てる

以下は、既存の一業務をエージェントへ渡すための導入順である。期限の保証ではない。接続や権限が不足するなら、その条件を解決する作業を先に置く。

1日目: 成果と現在地

最後に確認する外部結果を一文にする。現在はどこまでできていて、何が未確認かを書く。すでに別の自動処理が同じ仕事をしていないか、台帳と実行記録を読む。新しいループを先に作らない。

2日目: 一回を終える

最小の一件を、入力から結果まで完了させる。操作の途中で必要になった情報を記録する。自動化できない手続きが残れば、必要な条件を特定する。「人がやったはず」という推測で完了にしない。

3日目: 再開できる記録

実行後に別の担当が読むつもりで、所有者、対象ID、公式記録、次の一手を保存する。作業中の会話を読まなくても再開できるかを確かめる。全部のログを複製せず、必要な根拠へリンクする。

4日目: 二回目と重複防止

同じ対象を渡したときに、新しい請求や送信を繰り返さない条件を置く。安全に試せるfixtureや読み取りだけで、再実行の判定を確認する。本番への二重請求を検証手段にしない。

5日目: 失敗と回復

読み取り専用の観測や隔離した入力で、想定する失敗を一つ確認する。認証不足、古い入力、通信失敗、未確認の送信など、実際に起こりうるものを選ぶ。エラーを出すことより、次に何を取得するかが決まることを確かめる。

6日目: 定期起動

一回の仕事が通り、再開手順が残ってから定期起動へつなぐ。起動した記録と、業務を完了した記録を分ける。前の実行との競合、予算の上限、終了コードの扱いを確かめる。

7日目: 交代と数字

別の担当または新しい文脈で手順を読み、現在地を判定する。成果、費用、未確認を分けて報告する。次の一週間は一番詰まった境界を一つ直す。仕事を増やすのは、その後でよい。

12. 引き継げる会社にする

エージェントは停止する。モデルが変わる。実行環境が変わる。担当が使える時間や利用枠も変わる。交代できる設計は、失敗時の保険にとどまらず、日常の作業方法になる。

引き継ぎには、目的、正本、作業場所、ブランチ、使用したリリース、変更、検証、外部操作、未確認、次の一件を残す。正本や計画を一時フォルダだけに置かない。Gitで管理するものは、対象のコミットが共有先に存在するかまで確認する。

未確認の外部効果は、目立つ形で最初に書く。「送信ボタンを押したが送信済みを取得できていない。再送前に公式記録を取得する」のように、次の担当が重複させないための条件を明記する。

手順書は、コマンドの列だけでは足りない。何を得るために、どのアカウントと対象を使い、成功なら何が返り、失敗ならどこを見るかを書く。秘密値や一時的な認証URLは入れない。

引き継ぎを受けた担当は、正本を読み、対象の現在状態を確かめる。古い報告を無視して最初から始めることも、古い報告を無条件に現在の事実とすることも避ける。変わりやすい価格、認証、公開状態は、安く確認できるなら今の値を取る。

最後に、ルールを増やすときも小さくする。一回だけの固有事情は、そのタスクに残す。繰り返す失敗の種類が分かり、次に見るものまで説明できるときに、手順へ一般化する。長い常時指示で全員を縛る前に、いま進める仕事に必要な一行を選ぶ。

引き継ぎの受入: 次の担当が、前の担当へ「どこから続けるか」を聞かずに、同じ対象を重複操作せず、安全な次の一件へ進めること。

付録A. そのまま埋める一枚のタスク

> 目的：
> 現在確認できている事実：
> 今回の対象と範囲：
> 現在所有者：
> 実行してよい外部操作：
> 完了を示す公式記録：
> 未確認の外部操作：
> 保存する根拠：
> 次の一件：

最初に全部をきれいに埋めようとしない。分からないものは未確認と書き、その情報を取得する作業を次の一件にする。

付録B. 毎日の短い報告

> 確認した事実：
> 変更：
> 検証と実測結果：
> 残る故障・未確認：
> 次の一件：

「テスト成功」は何を確かめたのかを一言添える。「未払いではファイルが返らず、支払い済みの該当版だけが返ることを確認」のように、利用者への意味で説明する。

付録C. テンプレート版の内容

テンプレート版のZIPには、この本で使った考え方を、自分の運用へ移すための編集用ファイルを収録する。

- 役割契約: 目的、所有範囲、入力、出力、禁止事項。
- タスク正本: 現在地、完了条件、次の一件。
- 所有権台帳: 対象と現在の担当、実行経路。
- 実行イベント: 事実と未確認を分けるJSON例。
- 外部効果台帳: claim、送信、公式確認、再実行の扱い。
- 予算設定: 通貨、期間、上限、実測値、未取得の扱い。
- 商品契約: 版、価格、通貨、納品物、アクセス条件。
- 一回の実行手順: 起動から保存まで。
- 失敗対応票: 境界、追加観測、修正、再検証。
- 引き継ぎ票と日次報告。

これらは特定製品に依存しない文書と設定の雛形である。スケジューラ、認証機能、決済機能、実行エンジンそのものは含まない。ファイルを置くだけで業務が自動稼働するという意味ではない。

出典、制作範囲、利用条件

本書の運用構成は、Aniccaの役割、ループ、商品に関する内部台帳を読み、公開できる範囲へ一般化して作成する。内部ルールの存在と、すべての実行が常時それに従っていることは別である。私的な記録の原文、秘密値、顧客情報は公開しない。

販売の二段階構成では、Alex Ellis本人の「My first eBook - results and feedback」を参照する。本人の記事は、本文と実用ファイル等を分けたtier構成と、後日の売上報告を説明する。本書はその文章、商品内容、売上を流用せず、本文と雛形の違いを明確にする構造だけを参考にする。

https://blog.alexellis.io/my-first-ebook-results-feedback/

支払いリンクと購入後の確認には、利用する決済事業者の公式API契約を参照する。実装時には自分の環境で現行仕様を読み直す。

https://docs.stripe.com/payment-links/api

https://docs.stripe.com/api/payment-link/create

原稿とテンプレートのソースはAniccaの公開リポジトリで読める。有料版は整形したPDFおよびダウンロード用パッケージであり、ソースへの独占アクセスを売るものではない。

https://github.com/Daisuke134/anicca-products/tree/main/apps/landing/private/ebooks

本文と雛形は、購入者自身の個人・商用の運用へ編集して使える。未変更の本文やパッケージをそのまま再販売することは含めない。参照先の資料、サービス、商標の権利は各権利者に帰属する。実際の運用では、利用するサービスの契約、アカウントの所有権、権限、支出上限を確認する。